Who Wrote It? Identifying LLMs from Their Responses

Course: CMPSC 448 - Midterm Project Report
Team Leader: Jonghun Won
Team Members: Charlie Kim
GitHub Repository: https://github.com/cucu071212/CMPSC-448-Midterm.git
Submission Date: Oct 11, 2026

1. Problem Definition & Dataset Curation

As LLM-generated text spreads across public platforms, it is useful to know which model produced a given text. LLM families appear to differ in tone, structure and formatting, and we test whether those differences are enough for a classifier to identify the source model. We frame this as three-class classification among GPT, Claude and Gemini. We train a TextCNN and a bidirectional LSTM under the same pipeline and, for RQ2, give them the prompt only, the response only, or both.

We use the public LMArena arena-human-preference-140k dataset (135,634 conversations from April to July 2025, pinned revision 6322995a). In each record, two anonymous models answer the same user prompt, which gives us a natural control for RQ2. We keep English, text-only, single-turn conversations in which both models received the identical prompt, and each battle yields one sample per model. Models are grouped by ID prefix (gpt- and chatgpt- as GPT, claude- as Claude, gemini- as Gemini), and all other models are discarded. After removing duplicates, 53,682 responses remained: 25,366 from Claude, 18,317 from Gemini and 9,999 from GPT. Each sample has a prompt_id (a hash of the normalized prompt), LLM_Input, LLM_output, LLM_name and a label, with GPT = 0, Claude = 1 and Gemini = 2. The dataset was released publicly for research, with prompts under CC-BY-4.0 and responses under each provider's terms. We added no private data, and our repository does not redistribute the data.

Because the same prompt appears once per model, we split by prompt_id into 70% training, 15% validation and 15% test, and checked that no prompt appears in more than one split. We then downsampled each split to equal class sizes, which left 20,844 training samples (6,948 per family), 4,779 validation samples (1,593 per family) and 4,374 test samples (1,458 per family). Chance-level accuracy is therefore

33.3%. The cross-domain experiment (RQ3) uses a separate domain-based split, described in the RQ3 section.

Exploration showed large differences between families in response length, with medians of 243, 556 and 958 tokens for Claude, GPT and Gemini. Formatting differed as well. GPT uses emoji (27.5%) and em dashes (34.6%) far more than the others, and Gemini uses bold text most often (89.7%). Standard cleaning would remove these signals, so our tokenizer lowercases text but keeps punctuation, Markdown markers such as ** and ##, emoji and curly quotes as tokens, and maps newlines to <nl>. The vocabulary is built from the training split only and keeps tokens that appear at least twice, up to 30,000 entries. Sequences are truncated or padded to 512 tokens, a length that covers 52% of responses in full. For the “both” input, the first 128 prompt tokens are joined to the response with a <sep> token. Lowercasing and truncation do lose some information, such as capitalization habits and the ends of long responses.

2. Model Architecture & Training Details

We built two classifiers using PyTorch: a TextCNN and a bidirectional LSTM also known as a BiLSTM. Both models are trained to predict one of three large language model families: GPT, Claude and Gemini. The input text is converted to lowercase. Tokens are created while keeping punctuation and layout markers like symbols intact. The vocabulary is built from the training data. It has a size of 30,000 tokens and includes only tokens that appear at least twice. Sequences are truncated or right-padded to 512 tokens. Each token is represented by an embedding that is 128-dimensional.

CNN architecture

The TextCNN model uses three parallel one-dimensional convolutional layers. The kernel sizes are 3, 4 and 5. Each layer has 100 filters. After each convolution a ReLU activation function is applied, followed by max-over-time pooling. The pooling operation reduces each filter output to a value capturing the most prominent feature from that kernel. The three pooled vectors are then joined together into a vector of 300 dimensions. This is followed by a dropout layer with a dropout rate of 0.5 to reduce overfitting. Finally a connected layer produces three logits, one for each class. This design helps the model pick up on local patterns such as repeated phrases and formatting cues.

RNN architecture

The RNN uses a layer bidirectional LSTM with 128 hidden units in each direction. The forward pass and backward pass each produce a state vector. These two vectors are combined into a 256-dimensional representation. A dropout layer with a rate of 0.5 is applied before the classification layer. To handle variable-length sequences efficiently packed sequences are used. This avoids the model processing padding tokens during the steps. The BiLSTM captures context from both the beginning and the end of the sequence making it good at understanding dependencies across spans.

Training procedure

Both models are trained using cross-entropy loss. The optimizer used is AdamW with a learning rate of 0.001 and weight decay of 0.01. The batch size is set to 64. Training continues for up to 15 epochs. Gradient norms are clipped at 1.0 to stabilize training. A random seed of 42 is used to ensure reproducibility. After each epoch we evaluate the model on the validation set. Compute the macro-F1 score. The model checkpoint with the highest validation macro-F1 score is saved. Early stopping is triggered if there is no improvement in macro-F1 for three epochs. The best checkpoint is then used to make predictions on the test set.

Experimental setup

For research question RQ1 both models take the LLM response as input. For RQ2 we compare three input types: prompt-only, response-only, and prompt-plus-response. In the combined case up to 128 tokens from the prompt are followed by a separator token. Then the response is added. The total sequence length is capped at 512 tokens. This setup allows us to test how much information from the prompt helps in identifying the model family.

3. Results and how they are presented

We ran six experiments, training each model (CNN and RNN) under three input conditions: prompt only, response only, and prompt plus response. All runs used the same data splits, vocabulary settings, training loop and random seed (42). Each model was evaluated once on the test set, using the checkpoint with the best validation macro-F1. Table 1 summarizes the results. With 4,374 balanced test samples, the 95% margin of error is about ±0.8 percentage points for accuracies near 93% and about ±1.4 points near chance (33.3%).

Model	Input	Best epoch	Test Accuracy	Test macro-F1
CNN	Response only	10	93.14%	0.931
RNN	Response only	9	92.55%	0.925
CNN	Prompt only	11	32.99%	0.327
RNN	Prompt only	5	33.58%	0.335
CNN	Prompt + response	9	92.32%	0.923
RNN	Prompt + response	10	90.76%	0.908

Table 1. Test results of all six experiments.

For RQ1, both models identify the source family well from the response alone. The CNN reaches 93.14% accuracy (macro-F1 0.931) and the RNN 92.55% (0.925), far above the 33.3% chance level. The 0.59-point gap is within the margin of error, so we treat the two architectures as comparable in accuracy. The CNN was faster to train, though, at about 8 seconds per epoch on a T4 GPU against about 16 seconds for the RNN.

Table 2 breaks down the response-only results by family. Performance is balanced across the three classes, with every F1 score between 0.924 and 0.937. The CNN is most precise on GPT (0.950), so when it predicts GPT it is almost always right, although its recall on GPT is lower (0.925) and some GPT responses go to the other two classes. Claude shows the opposite pattern in both models. Its recall is higher than its precision (0.940 against 0.918 for the CNN, 0.931 against 0.918 for the RNN), meaning both models predict Claude more often than it actually occurs. Figure 1 shows the corresponding confusion matrices.

Model	Family	Precision	Recall	F1
CNN	GPT	0.950	0.925	0.937
CNN	Claude	0.918	0.940	0.929
CNN	Gemini	0.927	0.929	0.928
RNN	GPT	0.929	0.924	0.926
RNN	Claude	0.918	0.931	0.924
RNN	Gemini	0.930	0.921	0.926

Table 2. Per-class test results, response-only input.

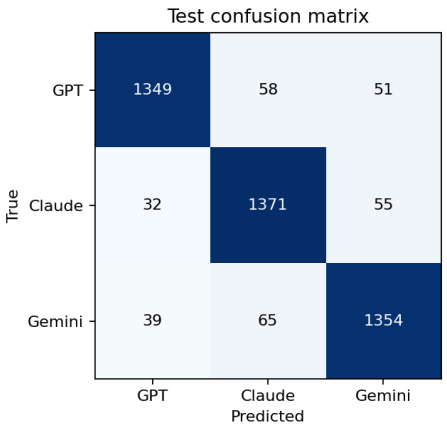


Figure 1. CNN training and confusion

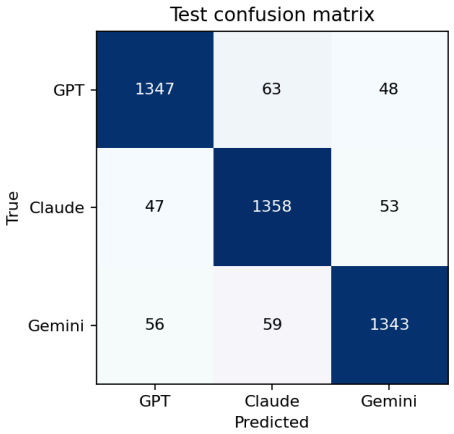


Figure 2. RNN training and confusion

For RQ2, models trained on the prompt alone performed at chance, with 32.99% for the CNN and 33.58% for the RNN, both within the margin of error around 33.3%. This is expected for Arena data. The two models in each battle receive exactly the same prompt, so the prompt says nothing about which model answered. The RNN's best validation score came as early as epoch 5, which fits the absence of anything to learn. Adding the prompt to the response did not help either. Accuracy dropped slightly for the CNN (from 93.14% to 92.32%) and more clearly for the RNN (from 92.55% to 90.76%). A likely reason is that the prompt takes up to 128 of the 512 input positions and leaves less room for the response, where the identifying signals are. The information that separates LLM families is in what the models write, and the user prompts contribute nothing to it.

4. In-Depth Analyses & Experiments

RQ1: Can we identify which LLM Generated a Response?

The first research question investigates whether the identity of the generating large language model can be inferred from its generated response. The results show that it is possible to identify the generating LLM from its output. Using only LLM-generated responses, the Convolutional Neural Network (CNN) achieved 93.14% test accuracy and a 0.9315 Macro-F1 score, while the Recurrent Neural Network (RNN) achieved 92.55% test accuracy and a 0.9255 Macro-F1 score. Both models performed well on the balanced three-class classification problem.

Looking at the performance of each class, the results show that classification was not driven by a single easily recognizable LLM. For the CNN, the F1 scores were 0.9375 for GPT, 0.9289 for Claude, and 0.9280 for Gemini. These consistent results across all three classes suggest that LLM-generated responses contain distinctive patterns that can be learned using neural text classifiers.

RQ2: Does the User's Prompt Help Identify the LLM?

To investigate whether user prompts improve LLM identification, we compared three input setups: input-only, output-only, and input + output.

First, when we used only user prompts, the CNN and RNN achieved 32.99% and 33.58% accuracy, respectively, which is close to the random-chance baseline of 33.33%. This suggests that user prompts alone provide little information about which LLM generated the response, reducing concerns regarding possible data bias related to prompt content.

Adding the prompt to the output decreased CNN accuracy from 93.14% to 92.32%, while RNN accuracy decreased from 92.55% to 90.76%.

Overall, our results suggest that generated responses are the primary source of information for LLM identification, while including user prompts did not improve classification performance for either model.

RQ3. Do LLM Fingerprints Generalize Across Tasks or Domains?

In order to address RQ3, we separated the Arena prompts into those relating to coding and those relating to math by means of case-insensitive keyword matching.

The coding keywords were "python", "javascript", "debug", and "sql", and the math keywords were "equation", "integral", "algebra", and "calculus". Prompts that matched both or neither of these categories were excluded.

In order to avoid data leakage we kept the prompt-grouped splits and ensured that all three families of LLMs were balanced. In each experiment 183 samples were used for training (61 for each family), 45 for validation (15 for each family) and the same coding test set of 210 samples (70 for each family) was employed. The Coding→Coding experiment used coding data for both training and validation, whereas the Math→Coding experiment used math data for both of these purposes. The models were trained from scratch using only the response text and had separate training vocabularies.

We kept the main CNN/RNN architectures and the settings for the optimizer, but limited the number of epochs to three, set the early-stopping patience to two, used a sequence length of 128, a batch size of 64, and a random seed of 42. The checkpoints were chosen on the basis of the validation macro-F1.

Model	Training Domain	Test Domain	Accuracy	Macro-F1
CNN	Coding	Coding	53.33%	0.5096
CNN	Math	Coding	50.48%	0.4425
RNN	Coding	Coding	65.71%	0.6487
RNN	Math	Coding	64.76%	0.6455

Table 3. In-Domain and Cross-Domain Classification Performance of CNN and RNN Models

In the in-domain scenario, the CNN reached an accuracy of 53.33% and a macro-F1 score of 0.5096. When it was trained on mathematical responses and then tested on coding responses, its accuracy fell to 50.48% and its macro-F1 score declined to 0.4425. Even though performance was lower in the cross-domain case, the small size of the test set means that it is difficult to determine whether the model depends more on domain-specific patterns.

In the in-domain scenario, the RNN reached an accuracy of 65.71% and a macro-F1 score of 0.6487. In the cross-domain case, its accuracy dropped slightly to 64.76%, although its macro-F1 score stayed almost the same at 0.6455.

In the case of both models, cross-domain accuracy stayed above the 33.3% level which would be expected by chance when the models were trained on mathematical answers and tested on coding answers. This indicates that certain patterns linked to LLM identity do transfer from one domain to another. Moreover, the RNN achieved better cross-domain performance than the CNN in this experiment.

Yet, since the size of the training set is small (183 samples), the coding test set is limited (210 samples), the sequence length is shorter, and only a single random seed is used, further experiments involving larger datasets, additional domains, and multiple random seeds would be required before one can draw any strong conclusions regarding cross-domain generalization.

RQ4. What Characteristics Distinguish Different LLMs?

To investigate what characteristics distinguish different LLMs, we analyzed response length, lexical diversity, and formatting patterns throughout GPT, Claude, and Gemini using 4,374 test samples.

First, we noticed the differences in response length. Gemini generated the longest responses, averaging 724.49 words, followed by GPT at 379.39 words and Claude at 258.33 words. This suggests that response length may provide useful information for identifying the LLM that generated the response.

Second, we compared formatting patterns, such as Markdown headings, bold text, and list items. GPT used Markdown headings in 64.20% of its responses, compared with 19.34% for Claude and 33.20% for Gemini. Meanwhile, Gemini used numbered lists in 60.91% of responses, significantly more frequently than GPT (28.67%) and Claude (31.55%).

Finally, we examined lexical diversity using the type-token ratio. Claude achieved the highest average ratio of 0.6373, followed by GPT (0.5495) and Gemini (0.4867). However, these differences may be influenced by response length, since longer texts generally have lower type-token ratios.

Overall, our data show that response length and formatting patterns are potential distinguishing characteristics of different LLM families. These differences may help explain the high classification accuracy achieved by our CNN and RNN models, although additional ablation experiments would be necessary to determine which features the classifiers actually rely on.

5. Lessons & Experience

Our first lesson was to test the pipeline on a subset before scaling up, and to make sure the subset limit is gone before the real run. Our first preprocessing run scanned only 20,000 of the 135,634 Arena battles, because we had left in a limit meant for quick testing. That gave us 1,007 training examples per family and a 693-response test set. With the full dataset we got 6,948 per family and 4,374 test responses, and the accuracy uncertainty dropped from about ±3.7 to ±1.4 points.

We also found that data exploration should come before preprocessing decisions. Before building the loader, we found large differences in response length (median 243, 556 and 958 tokens for Claude, GPT and Gemini) and in formatting habits like Markdown, emoji, em dashes and curly apostrophes. Standard preprocessing would have stripped these signals out, so we kept them as tokens and set the maximum length to 512 based on the length distribution. Looking at the data also exposed a subtle bug: pandas read one response, “NA”, as a missing value. We only noticed because the row count was off by one.

Finally, a fair comparison between the two models depended on a single shared pipeline. The CNN and RNN use the same tokenizer, vocabulary, training loop and evaluation, so any difference in results comes from the architecture. At one point we had two separate pipelines, and their results would not have been comparable. Since then we agree on the interfaces (column names, label ids, tensor shapes) before anyone writes model code.